

1. [7 points] Consider the **Library Issue Records** table given below:

StudentID	BookID	BookTitle	IssueDate	ReturnDate
S01	B03	Glens' Book	2024-03-10	2024-03-20
S07	B12	Grims' Book	2024-03-09	2024-03-18
S04	B24	Goals' Book	2024-03-10	2024-03-25
S56	B09	Gems' Book	2024-03-09	2024-03-22
S74	B15	Gulfs' Book	2024-03-10	2024-03-20
S72	B06	Globs' Book	2024-03-09	2024-03-19
S04	B03	Gulds' Book	2024-03-10	2024-03-20
S03	B13	Gangs' Book	2024-03-18	2024-03-24
S02	B04	Gales' Book	2024-03-20	2024-03-20
S71	B21	Grays' Book	2024-03-22	2024-03-23

- (a) Identify the smallest possible Candidate Key(s) & Primary Key(s) for the table with the filled data above. Assume that this is the complete table and there is no new data to be added. Clearly justify your choices based on the data. [2+2]
- (b) Suppose the dataset contains a total of **five attributes**, and among them, exactly **three attributes are unique** (i.e., each can uniquely identify a tuple on its own). Determine the **total number of possible Super Keys** that can be formed from these attributes. Provide your reasoning clearly. [3]

(a) ~~Primary~~ Primary Keys : BookTitle, {StudentID, BookID} (Minimal Candidate Key)

StudentID : 104, 104 X

Book ID : B03, B03 X

IssueDate & ReturnDate : X

Smallest possible PK : BookTitle

Candidate Keys : {BookTitle, StudentID}, {BookTitle, BookID}, ...,
 {BookTitle, StudentID, BookID}, {BookTitle, BookID, IssueDate}, ...,
 {BookTitle, StudentID, BookID, IssueDate}, ...,
 {BookTitle, StudentID, BookID, IssueDate, ReturnDate}, {BookTitle},
 {StudentID, BookID}

Smallest Possible Candidate Key : BookTitle

(b) Total no. of attributes : 5 (A, B, C, D, E) ← example

No. of Unique attributes : 3 (A, B, C) ← Example

Super Keys : A, (1x⁴C₄)

AB, AC, AD, AE, (1x⁴C₄)

ABC, ABD, ABE, ACD, ACE, ADE (1x⁴C₃)

ABCD, ABCE, ABDE, ACDE (1x⁴C₂)

ABCD E (1x⁴C₁)

(2⁴) ← Total no. of supersets

B,

BC, BD, BE, (BA repeated)

BCD, BCE, BDE

BCDE

(2³)

C,

CD, CE,

COE

(2²)

3! MCE.

Total no. of Super Keys : 2⁴ + 2³ + 2²

In General, if Total no. of attributes = n, Total no. of Unique attributes = k

$$\text{Total no. of Super Keys} = 2^{n-1} + 2^{n-2} + \dots + 2^{n-k} = \frac{2^{n-1}((\frac{1}{2})^k - 1)}{(\frac{1}{2}) - 1} = 2^n(1 - \frac{1}{2^k})$$

2. [10 points] Consider the following two relational schemas for a university system:

Table: Student

Column Name	Description and Type Hints
StudentID	Unique student identifier (exactly 5 characters)
StudentName	Full name of the student (up to 50 characters)
Age	Age of the student (must be 18 or above)
Gender	Gender of the student - 'M', 'F', or 'O' (single character; default is 'O')
CourseID	ID of the course(s) enrolled (exactly 5 characters)
JoinDate	Date when the student joined (date format)
Email	Email address of the student (unique and upto 100 characters)

Table: Course

Column Name	Description and Type Hints
CourseID	Unique course code (exactly 5 characters)
CourseName	Title of the course (up to 100 characters)
Credits	Number of credits (integer value between 1 and 5)
Department	Department offering the course (up to 30 characters)
Instructor	Name of the course instructor (up to 50 characters)

Write the complete SQL CREATE TABLE statements for both the Student and Course tables. Ensure that you choose the most appropriate data types and constraints wherever applicable. Include suitable ON DELETE actions. Clearly state any assumptions made.

```
CREATE TABLE Student (
  0.5 StudentID CHAR(5) PRIMARY KEY,
  0.5 StudentName VARCHAR(50)
  0.5 Age INT NOT NULL,
  0.5 CHECK (Age ≥ 18),
  0.5 Gender CHAR(1) DEFAULT 'O',
  0.5 CHECK (Gender IN {'M', 'F', 'O'}),
  0.5 CourseID CHAR(5),
  0.5 JoinDate DATE NOT NULL
  DEFAULT '2024-08-1',
  0.5 Email VARCHAR(100) PRIMARY KEY,
  0.5 );
```

Assumptions:

- Course name cannot be NULL, Department also cannot be NULL.
- Age & Gender cannot be NULL
- If a course is deleted, we still want access to ~~those~~ the info of who has enrolled & taught in the past. ∴ No Cascade.

```
CREATE TABLE Course (
```

```
0.5 CourseID CHAR(5) PRIMARY KEY,
0.5 CourseName VARCHAR(100) NOT NULL,
0.5 Credits INT,
0.5 CHECK (Credits IN {1, 2, 3, 4, 5}),
0.5 Department VARCHAR(30) NOT NULL,
0.5 Instructor VARCHAR(50),
FOREIGN KEY CourseID REFERENCES
  Course (CourseID)
  Student.CourseID ON DELETE
  SET NULL
);
```

5.5/6

4

-0.25 FK

-0.25 check credits
3.5/6.4 (9)

3. [3 points] From the previous schema that you have made, write an SQL query to print the Student Name of all the Female (F) students whose age is less than 30 and either takes the course with ID "DNA25" or "RSI25".

```

SELECT S.StudentName
FROM student AS S
WHERE S.Gender = 'F' AND S.Age < 30
AND CourseID in ('DNA25', 'RSI25');

```

4. [10 points] In a quiet town nestled between rolling hills, two friends, George and Pratishttha, decided to bring an old, forgotten community library back to life. They named it "The Library of Precog". Pratishttha, with her meticulous planning skills, organized the books, while George, a budding programmer, took on the challenge of building a database to manage their small but growing collection.

After weeks of design, George finalized the database structure with two core tables: Books and Borrows. He was particularly proud of the constraints he put in place. "This will keep our data clean and reliable!" he declared. He defined primary keys to ensure every book and every loan was unique, foreign keys to link loans back to specific books, and check constraints to prevent invalid data from ever being entered. He also added 'NOT NULL' and 'UNIQUE' constraints where necessary to maintain data integrity.

Table: Books

Column Name	Description and Constraints
BookID	Unique identifier for each book; serves as the primary key (cannot be null).
Title	Title of the book; must be provided and cannot be null; should be unique.
Author	Name of the author; required field (cannot be null).
Genre	Category or type of the book (optional field).
Status	Current availability status of the book; can be either 'Available' or 'Borrowed'; defaults to 'Available' if not specified.

BookID	Title	Author	Genre	Status
101	The Martian	Andy Weir	Science Fiction	Borrowed
102	Project Hail Mary	Andy Weir	Science Fiction	Available
103	Circe	Madeline Miller	Fantasy	Borrowed
104	The Silent Patient	Alex Michaelides	Thriller	Available
105	Dune	Frank Herbert	Science Fiction	Available

Table: Borrows

Column Name	Description and Constraints
BorrowID	Unique identifier for each borrowing record; serves as the primary key (cannot be null).
BookID	Identifier of the borrowed book; references the corresponding BookID in the Books table;
BorrowerName	Name of the person who borrowed the book; required field (cannot be null).
BorrowDate	Date on which the book was issued (optional; stores date values).
DueDate	Date by which the book must be returned (optional; stores date values).

BorrowID	BookID	BorrowerName	BorrowDate	DueDate
1	101	George	2025-10-15	2025-11-15
2	103	Pratishtha	2025-10-20	2025-11-20

For each independent SQL statement below, identify if there are any violated constraints. Consider default constraints and actions where not specified. Justify your answer.

1. INSERT INTO Books (BookID, Title, Author, Genre, Status) VALUES (106, 'The Martian', 'Andy Weir', 'Science Fiction', 'Available');

Title cannot be duplicate, but Already BookID 101 has 'The Martian'.

∴ Key constraint violation

✓

[∴ Title → Unique] ✓

2. INSERT INTO Books (BookID, Title, Genre, Status) VALUES (107, 'A new book', 'Mystery', 'Available');

Author field is required, cannot be NULL

∴ Entity Integrity constraint

✓

Author field also does not have default value (if yes, could have left blank)

3. UPDATE Books SET Status = 'On Hold' WHERE BookID = 103;

Status attribute can take values only 'Available' & 'Borrowed'.

'On Hold' is not in the domain of Status Attribute.

✓

∴ Domain constraint ~~is~~ Violation

4. UPDATE Borrows SET BookID = 105, BorrowID = 1 WHERE BorrowerName = 'George';

There already exists an entity with BookID = 105 in Books table.

Hence updating results in an error (SET BookID = 105)

✓

∴ This is a foreign key, this comes under

Referential Integrity Constraints.

5. DELETE FROM Books WHERE BookID = 103;

There are no ON DELETE REJECT / SET NULL / CASCADE information for provided Referencing table.

∴ If an entry is deleted in Books Table, the Borrows table having BookID = 103 wouldn't make sense.

✓

∴ Referential Integrity Constraint

✓ (NO ON DELETE operation mentioned)